

Trivia Attack — Esqueletos Java

A continuación encontrarás un conjunto de archivos Java (esqueleto) organizados por paquetes. Están pensados para compilar como base de diseño: clases, interfaces, patrones (Estado, Iterador, Fábrica Abstracta, Estrategia) y comentarios `TODO` donde implementar lógica de juego y sincronización (hilos, locks).

com.trivia.model

```
package com.trivia.model;

public enum Categoria {Arte, Ciencia, Deporte, Entretenimiento, Geografia,
Historia}

public enum Rareza {Comun, Inusual, Rara, Legendaria}
```

```
package com.trivia.model;

public class Respuesta {
    private final String texto;
    private final boolean correcta;

    public Respuesta(String texto, boolean correcta) {
        this.texto = texto;
        this.correcta = correcta;
    }

    public String getTexto() { return texto; }
    public boolean esCorrecta() { return correcta; }
}
```

```
package com.trivia.model;

public class Pregunta {
    private final int id;
    private final Categoria categoria;
    private final String texto;
    private final Respuesta[] respuestas; // tamaño 4 por invariante

    public Pregunta(int id, Categoria categoria, String texto, Respuesta[]
respuestas) {
```

```

        if (respuestas == null || respuestas.length != 4) throw new
IllegalArgumentException("Debe haber 4 respuestas");
        this.id = id;
        this.categoria = categoria;
        this.texto = texto;
        this.respuestas = respuestas;
    }

    public int getId() { return id; }
    public Categoria getCategoria() { return categoria; }
    public String getTexto() { return texto; }
    public Respuesta[] getRespuestas() { return respuestas; }
}

```

com.trivia.util

```

package com.trivia.util;

import java.util.Random;

public class Dado {
    private final int numeroCaras;
    private int resultado;
    private final Random rnd = new Random();

    public Dado(int numeroCaras) {
        if (numeroCaras < 2) throw new IllegalArgumentException("Datos deben
tener al menos 2 caras");
        this.numeroCaras = numeroCaras;
    }

    public int tirar() {
        resultado = rnd.nextInt(numeroCaras) + 1;
        return resultado;
    }

    public int getResultado() { return resultado; }
}

```

com.trivia.cards (jerarquía de cartas y patrón Estrategia)

```
package com.trivia.cards;

import com.trivia.model.Rareza;

public abstract class CartaPoder {
    protected final int id;
    protected final Rareza rareza;
    protected final int espacioEnInventario;
    protected boolean jugada = false;
    protected boolean congelada = false;
    protected boolean utilizable = true;

    protected final com.trivia.cards.strategy.CardEffect efecto;

    public CartaPoder(int id, Rareza rareza, int espacio,
com.trivia.cards.strategy.CardEffect efecto) {
        this.id = id;
        this.rareza = rareza;
        this.espacioEnInventario = espacio;
        this.efecto = efecto;
    }

    public int getId() { return id; }
    public Rareza getRareza() { return rareza; }
    public int getEspacioEnInventario() { return espacioEnInventario; }

    public boolean isJugada() { return jugada; }
    public void setJugada(boolean jugada) { this.jugada = jugada; }

    public boolean isCongelada() { return congelada; }
    public void setCongelada(boolean congelada) { this.congelada = congelada; }

    public boolean isUtilizable() { return utilizable; }
    public void setUtilizable(boolean u) { this.utilizable = u; }

    // Strategy: delega el efecto concreto a implementaciones de CardEffect
    public void usar(com.trivia.game.Partida partida, com.trivia.player.Jugador
actor) {
        if (!utilizable) throw new
IllegalStateException("Carta no utilizable en este momento");
        efecto.apply(partida, actor, this);
        this.jugada = true;
    }
}
```

```

package com.trivia.cards;

import com.trivia.model.Rareza;

public class CartaPoderInmediata extends CartaPoder {
    private final String nombre;
    private final String descripcion;

    public CartaPoderInmediata(int id, Rareza rareza, int espacio, String
nombre, String descripcion, com.trivia.cards.strategy.CardEffect efecto) {
        super(id, rareza, espacio, efecto);
        this.nombre = nombre;
        this.descripcion = descripcion;
    }

    public String getNombre() { return nombre; }
    public String getDescripcion() { return descripcion; }
}

```

```

package com.trivia.cards;

import com.trivia.model.Rareza;

public class CartaPoderAmbiental extends CartaPoder {
    private final String nombre;
    private final String descripcion;
    private boolean activa = false;

    public CartaPoderAmbiental(int id, Rareza rareza, int espacio, String
nombre, String descripcion, com.trivia.cards.strategy.CardEffect efecto) {
        super(id, rareza, espacio, efecto);
        this.nombre = nombre;
        this.descripcion = descripcion;
    }

    public void setActiva(boolean a) { this.activa = a; }
    public boolean isActiva() { return activa; }
}

```

```

package com.trivia.cards;

import com.trivia.model.Rareza;

public class CartaPoderInventario extends CartaPoder {
    private final String nombre;

```

```

        private final String descripcion;

        public CartaPoderInventario(int id, Rareza rareza, int espacio, String
nombre, String descripcion, com.trivia.cards.strategy.CardEffect efecto) {
            super(id, rareza, espacio, efecto);
            this.nombre = nombre;
            this.descripcion = descripcion;
        }
    }
}

```

```

package com.trivia.cards;

import com.trivia.model.Rareza;

public class CartaPoderFragil extends CartaPoder {
    private final String nombre;
    private final String descripcion;
    private boolean rota = false;

    public CartaPoderFragil(int id, Rareza rareza, int espacio, String nombre,
String descripcion, com.trivia.cards.strategy.CardEffect efecto) {
        super(id, rareza, espacio, efecto);
        this.nombre = nombre;
        this.descripcion = descripcion;
    }

    public boolean isRota() { return rota; }
    public void setRota(boolean rota) { this.rota = rota; }
}

```

```

package com.trivia.cards.strategy;

import com.trivia.cards.CartaPoder;
import com.trivia.game.Partida;
import com.trivia.player.Jugador;

/** Strategy: cada carta define su efecto implementando apply */
@FunctionalInterface
public interface CardEffect {
    void apply(Partida partida, Jugador actor, CartaPoder carta);
}

```

```

package com.trivia.cards.strategy.impl;

```

```

import com.trivia.cards.CartaPoder;
import com.trivia.cards.strategy.CardEffect;
import com.trivia.game.Partida;
import com.trivia.player.Jugador;

/** Ejemplo simple: ganar 1 punto al actor */
public class EfectoGanaUnPunto implements CardEffect {
    @Override
    public void apply(Partida partida, Jugador actor, CartaPoder carta) {
        // TODO: la puntuación está en Partida; actualizarla con sincronización
        partida.incrementarPuntos(actor, 1);
    }
}

```

com.trivia.piles (Montones e Iteradores)

```

package com.trivia.piles;

import com.trivia.model.Pregunta;
import com.trivia.model.Categoria;

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class MontonPreguntas implements Iterable<Pregunta> {
    private final Categoria categoria;
    private final List<Pregunta> preguntas = new ArrayList<>();

    public MontonPreguntas(Categoria categoria) {
        this.categoria = categoria;
    }

    public Categoria getCategoria() { return categoria; }
    public void addPregunta(Pregunta p) { preguntas.add(p); }

    @Override
    public Iterator<Pregunta> iterator() {
        return preguntas.iterator();
    }

    public Pregunta robar() {
        if (preguntas.isEmpty()) return null;
        return preguntas.remove(0);
    }
}

```

```
}  
}
```

```
package com.trivia.piles;  
  
import com.trivia.cards.CartaPoder;  
  
import java.util.ArrayList;  
import java.util.Iterator;  
import java.util.List;  
  
public class MontonCartas implements Iterable<CartaPoder> {  
    private final List<CartaPoder> cartas = new ArrayList<>();  
  
    public void addCarta(CartaPoder c) { cartas.add(c); }  
    public CartaPoder robar() { return cartas.isEmpty() ? null :  
cartas.remove(0); }  
    @Override  
    public Iterator<CartaPoder> iterator() { return cartas.iterator(); }  
}
```

com.trivia.player (Jugador y Patrón Estado + Hilo)

```
package com.trivia.player;  
  
import com.trivia.model.Categoria;  
import java.util.Arrays;  
import java.util.concurrent.atomic.AtomicInteger;  
  
/** Estadísticas permanentes del jugador */  
public class Jugador {  
    private final int id;  
    private final String nombre;  
    // orden alfabético: Arte, Ciencia, Deporte, Entretenimiento, Geografia,  
Historia  
    private final AtomicInteger[] historialEfectividad = new AtomicInteger[6];  
  
    public Jugador(int id, String nombre) {  
        this.id = id;  
        this.nombre = nombre;  
        for (int i=0;i<6;i++) historialEfectividad[i]=new AtomicInteger(0);  
    }  
}
```

```

    public int getId() { return id; }
    public String getNombre() { return nombre; }

    public void aumentarEfectividad(Categoria c, int delta) {
        historialEfectividad[c.ordinal()].addAndGet(delta);
    }

    public int getEfectividad(Categoria c) { return
        historialEfectividad[c.ordinal()].get(); }
}

```

```

package com.trivia.player;

import com.trivia.game.Turno;

/** Estado del jugador: Patrón Estado */
public abstract class EstadoJugador {
    protected final Jugador jugador;

    public EstadoJugador(Jugador jugador) { this.jugador = jugador; }

    public abstract void onTurnStart();
    public abstract void onTurnEnd();
    public abstract void recibirPregunta(Turno turno);
}

```

```

package com.trivia.player;

import com.trivia.game.Turno;

public class Preguntador extends EstadoJugador {
    public Preguntador(Jugador j) { super(j); }
    @Override public void onTurnStart() { /* TODO: acciones al iniciar turno */}
    @Override public void onTurnEnd() { /* TODO */}
    @Override public void recibirPregunta(Turno turno) { throw new
        IllegalStateException("No puedes recibir pregunta si eres preguntador"); }
}

```

```

package com.trivia.player;

import com.trivia.game.Turno;

public class Respondedor extends EstadoJugador {
    public Respondedor(Jugador j) { super(j); }
}

```

```

@Override public void onTurnStart() { /* normalmente no hace nada */ }
@Override public void onTurnEnd() { /* TODO */ }
@Override public void recibirPregunta(Turno turno) { /* responder la
pregunta del turno */ }
}

```

```

package com.trivia.player;

import com.trivia.game.Turno;

public class EnEspera extends EstadoJugador {
    public EnEspera(Jugador j) { super(j); }
    @Override public void onTurnStart() { }
    @Override public void onTurnEnd() { }
    @Override public void recibirPregunta(Turno turno) { /* aceptar o recibir
preguntas */ }
}

```

```

package com.trivia.player;

import com.trivia.game.Partida;

/** Hilo asociado a cada jugador para permitir acciones fuera de su turno */
public class PlayerThread implements Runnable {
    private final Jugador jugador;
    private final Partida partida;
    private volatile boolean running = true;

    public PlayerThread(Jugador jugador, Partida partida) {
        this.jugador = jugador;
        this.partida = partida;
    }

    public void stop() { running = false; }

    @Override
    public void run() {
        while (running) {
            // TODO: esperar notificaciones del servidor/partida y permitir usar
            cartas fuera de turno
            try { Thread.sleep(200); } catch (InterruptedException e) {
                Thread.currentThread().interrupt(); }
        }
    }
}

```

com.trivia.game (Partida, Turno)

```
package com.trivia.game;

import com.trivia.model.Pregunta;
import com.trivia.player.Jugador;
import com.trivia.cards.CartaPoder;

public class Turno {
    public final Jugador jugadorPreguntador;
    public final Jugador jugadorRespondedor;
    public final Pregunta pregunta;
    public String respuestaRespondedor; // letra o texto
    public int puntosAsignados;
    public int temporizador; // segundos

    public Turno(Jugador preguntador, Jugador respondedor, Pregunta pregunta,
int temporizador) {
        this.jugadorPreguntador = preguntador;
        this.jugadorRespondedor = respondedor;
        this.pregunta = pregunta;
        this.temporizador = temporizador;
    }
}
```

```
package com.trivia.game;

import com.trivia.model.MontonPreguntas; // si está en otro paquete, ajustar
import
import com.trivia.model.Pregunta;
import com.trivia.player.Jugador;
import com.trivia.cards.CartaPoder;
import com.trivia.piles.MontonPreguntas as Pile; // placeholder: ajustar según
organización real

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Partida {
    private int vueltaActual = 1;
    private final List<Turno> historialTurnos = new ArrayList<>();
    private final List<com.trivia.piles.MontonPreguntas> montonesPreguntas =
new ArrayList<>(6);
```

```

private final List<Jugador> jugadores = new ArrayList<>();
private final List<Integer> puntuacionJugadores = new ArrayList<>();
private final List<Integer> preguntasGanadasJugadores = new ArrayList<>();
private final List<Integer> tamanosInventario = new ArrayList<>();
private final List<List<CartaPoder>> inventarios = new ArrayList<>();
private final List<CartaPoder> cartasActivas = new ArrayList<>();
private final List<Pregunta> yaPreguntadas = new ArrayList<>();
private final List<Boolean> listos = new ArrayList<>();

public Partida(List<Jugador> jugadores) {
    if (jugadores.size() < 3 || jugadores.size() > 6) throw new
IllegalArgumentException("Número de jugadores inválido");
    this.jugadores.addAll(jugadores);
    for (int i=0;i<jugadores.size();i++) {
        puntuacionJugadores.add(3); // puntuación inicial según
especificación
        preguntasGanadasJugadores.add(0);
        tamanosInventario.add(2);
        inventarios.add(new ArrayList<>());
        listos.add(false);
    }
}

// Operaciones de ejemplo (sincronizar en implementación real):
public synchronized void incrementarPuntos(Jugador j, int delta) {
    int idx = jugadores.indexOf(j);
    if (idx < 0) return;
    puntuacionJugadores.set(idx, puntuacionJugadores.get(idx) + delta);
}

public synchronized void registrarPreguntaUsada(Pregunta p) {
    yaPreguntadas.add(p); }

public int getVueltaActual() { return vueltaActual; }
public void siguienteTurno() { /* TODO: lógica de rotación y control de
vuelta */ }

// TODO: métodos para robar cartas de montones, gestionar cartas activas,
inventarios, temporizadores, etc.
}

```

com.trivia.cards.factory (Fábrica Abstracta)

```
package com.trivia.cards.factory;

import com.trivia.cards.CartaPoder;
import com.trivia.piles.MontonCartas;

public abstract class AbstractCardFactory {
    public abstract MontonCartas crearMontonCompleto();
    public abstract CartaPoder crearCartaPorId(int id);
}

package com.trivia.cards.factory;

import com.trivia.cards.CartaPoder;
import com.trivia.piles.MontonCartas;
import com.trivia.model.Rareza;
import com.trivia.cards.strategy.impl.EfectoGanaUnPunto;

public class DefaultCardFactory extends AbstractCardFactory {
    @Override
    public MontonCartas crearMontonCompleto() {
        MontonCartas m = new MontonCartas();
        // TODO: poblar con cartas concretas
        return m;
    }

    @Override
    public CartaPoder crearCartaPorId(int id) {
        // TODO: switch/registro de cartas según id
        return null;
    }
}
```

Notas sobre concurrencia y hilos

- La intención es que cada jugador tenga un `PlayerThread` que escuche eventos y permita usar cartas fuera del turno. La `Partida` debe exponer métodos sincronizados o usar `java.util.concurrent.locks` para coordinar acceso a inventarios y cartas.
 - Evitar condiciones de carrera al modificar `cartasActivas`, `inventarios` o `puntuacionJugadores`.
-

Próximos pasos sugeridos

1. Elegir un subconjunto de cartas concretas a implementar y modelar sus `CardEffect`.
2. Implementar la lógica de `Partida.siguienteTurno()` y el mecanismo de temporizadores.
3. Crear tests unitarios para invariantes (p. ej. que no se repitan preguntas dentro de la misma `Partida`).

Fin del esqueletado.

Invariantes reflejadas como `assert` (añadido)

A continuación se incluyen fragmentos de código con `assert` Java que reflejan las invariantes OCL del modelo. Inserta o integra estos fragmentos en las clases señaladas (constructores o setters) para que las comprobaciones se ejecuten en tiempo de ejecución cuando las aserciones estén activadas (`-ea`).

Partida (constructor / control de vueltas)

```
// En constructor de Partida:
assert jugadores.size() >= 3 && jugadores.size() <= 6 : "NumJugadoresEntre3Y6:
debe haber entre 3 y 6 jugadores";

// Cuando se actualice la vuelta o al inicio de cada turno
public void setVueltaActual(int v) {
    assert v >= 1 && v <= 6 : "PartidaMaximo6Vueltas: vuelta fuera de rango";
    this.vueltaActual = v;
}
```

Pregunta (constructor)

```
public Pregunta(int id, Categoria categoria, String texto, Respuesta[]
respuestas) {
    assert respuestas != null : "Respuestas no puede ser null";
    assert respuestas.length == 4 :
"PreguntaTieneCuatroRespuestas: debe haber 4 respuestas";
    this.id = id;
    this.categoria = categoria;
    this.texto = texto;
    this.respuestas = respuestas;
}
```

MontonPreguntas (al agregar pregunta)

```
public void addPregunta(Pregunta p) {
    assert p != null : "No se puede añadir pregunta null";
    assert p.getCategoria() == this.categoria : "MismaCategoría: la pregunta debe pertenecer al mismo montón";
    preguntas.add(p);
}
```

Dado (tirar)

```
public int tirar() {
    resultado = rnd.nextInt(numeroCaras) + 1;
    assert resultado >= 1 && resultado <= numeroCaras :
    "DadoDaResultadoCorrecto: resultado fuera de rango";
    return resultado;
}
```

CartaPoderInmediata (constructor)

```
public CartaPoderInmediata(int id, Rareza rareza, int espacio, String nombre,
String descripcion, com.trivia.cards.strategy.CardEffect efecto) {
    super(id, rareza, espacio, efecto);
    this.nombre = nombre;
    this.descripcion = descripcion;
    // Invariante: las inmediatas se juegan al obtenerlas
    this.jugada = true;
    assert this.jugada : "InmediataJugada: las cartas inmediatas deben marcarse como jugadas";
}
```

CartaPoderAmbiental (no en inventario y activas mientras figura en cartasActivas)

```
// Al activar/crear:
public CartaPoderAmbiental(...) {
    // ...constructor
    this.activa = true; // cuando se juega
}

// En Partida: al limpiar cartas activas se debe comprobar
public synchronized void limpiarCartaAmbiental(CartaPoderAmbiental c) {
    c.setActiva(false);
    cartasActivas.remove(c);
}
```

```

        assert !cartasActivas.contains(c) || c.isActiva() :
"CartasAmbientalesActivas: las cartas ambientales en cartasActivas deben estar
activas";
    }

    // Además, en comprobación general se puede añadir:
    private void checkCartasAmbientales() {
        for (CartaPoder cp : cartasActivas) {
            if (cp instanceof CartaPoderAmbiental) {
                assert ((CartaPoderAmbiental) cp).isActiva() :
"CartasAmbientalesActivas: ambientales en lista deben estar activas";
            }
        }
    }
}

```

Preguntas únicas (registro)

```

public synchronized void registrarPreguntaUsada(Pregunta p) {
    assert p != null;
    for (Pregunta q : yaPreguntadas) {
        assert q.getId() != p.getId() : "PreguntasUnicas: la pregunta ya fue
usada en la partida";
    }
    yaPreguntadas.add(p);
}

```

Cartas frágiles fuera del inventario cuando rotas

```

public synchronized void eliminarCartasFragilesRota(int jugadorIndex) {
    List<CartaPoder> inv = inventarios.get(jugadorIndex);
    inv.removeIf(c -> (c instanceof CartaPoderFragil) && ((CartaPoderFragil)
c).isRota());
    // Invariante: tras esta operación no debe quedar ninguna carta frágil rota
    for (CartaPoder c : inv) {
        assert !(c instanceof CartaPoderFragil && ((CartaPoderFragil)
c).isRota()) : "CartasFragilesRotasFueraInventario";
    }
}

```

Cartas ambientales y cartas inmediatas no en inventarios

```

public void checkInventariosNoContienenAmbientalesNiInmediatas() {
    for (List<CartaPoder> inv : inventarios) {
        for (CartaPoder c : inv) {

```

```

        assert !(c instanceof com.trivia.cards.CartaPoderAmbiental) :
"CartasAmbientalesNoEnInventario";
        assert !(c instanceof com.trivia.cards.CartaPoderInmediata) :
"CartasInmediatasNoEnInventario";
    }
}
}

```

CartaPoderEterea (espacio en inventario = 0)

```

public class CartaPoderEterea extends CartaPoder {
    public CartaPoderEterea(int id, Rareza rareza, String nombre, String
descripcion, com.trivia.cards.strategy.CardEffect efecto) {
        super(id, rareza, 0, efecto); // espacio en inventario = 0
        // invariante explícita
        assert this.espacioEnInventario == 0 :
"EtereaNoOcupa: cartas etéreas no deben ocupar inventario";
    }
}

```

Turno: el preguntador no puede ser el mismo que el respondedor

```

public Turno(Jugador preguntador, Jugador respondedor, Pregunta pregunta, int
temporizador) {
    assert preguntador != null && respondedor != null :
"Jugadores no pueden ser null";
    assert !preguntador.equals(respondedor) :
"PreguntadorDistintoDeRespondedor: no puede preguntarse a si mismo";
    this.jugadorPreguntador = preguntador;
    this.jugadorRespondedor = respondedor;
    this.pregunta = pregunta;
    this.temporizador = temporizador;
}

```

Nota final

- Estas `assert` deben considerarse complementarias a validaciones y excepciones que necesites en tiempo de ejecución en producción. Las aserciones de Java se suelen activar en tests y entornos de desarrollo con `-ea`, por lo que en producción pueden estar desactivadas. Si deseas que las comprobaciones sean siempre activas, conviene lanzar `IllegalArgumentException` o `IllegalStateException` en lugar de `assert`.

Si quieres, aplico estos cambios directamente al código del canvas (integrándolos en cada clase ya presente) y genero un commit con los cambios. ¿Lo hago? Si prefieres que aplique solo a clases concretas, dime cuáles y lo haré inmediatamente.